

Prueba técnica Mobile – Droide de combate YVH

Contexto

Estimado/a Comandante,

La Nueva República está finalizando el desarrollo del droide de combate **YVH**, al que aún le falta el módulo de **selección de objetivos**.

Durante esta prueba, el droide operará en **modo entrenamiento**, por lo que el módulo de visión proporcionará los datos de escaneo de forma local.

Tu misión será desarrollar una **aplicación mobile** que implemente la lógica necesaria para determinar **cuál es el siguiente objetivo a atacar**, aplicando una serie de protocolos tácticos.

El foco principal de la prueba es: modelado de dominio, lógica, arquitectura y testabilidad. La interfaz puede ser sencilla, pero funcional.

Datos de entrada

El módulo de visión proporciona un objeto JSON con la siguiente estructura:

```
{
  "protocols": ["avoid-mech"],
  "scan": [
    {
      "coordinates": { "x": 0, "y": 40 },
      "enemies": { "type": "soldier", "number": 10 }
    }
  ]
}
```

Campos

- **protocols**
Lista de protocolos que deben aplicarse para determinar el objetivo a atacar.
- **scan**
Lista de puntos detectados por el radar.

Cada punto contiene:

- **coordinates:** coordenadas `x` e `y`
 - **enemies:**
 - `type: "soldier"` o `"mech"`
 - `number:` número de enemigos
 - **allies** (opcional): número de aliados presentes en la posición
-

Resultado esperado

La aplicación debe devolver las coordenadas del siguiente objetivo a atacar:

```
{  
  "x": 0,  
  "y": 40  
}
```

Protocolos disponibles

- **closest-enemies:** Priorizar el punto más cercano con enemigos.
- **furthest-enemies:** Priorizar el punto más lejano con enemigos.
- **assist-allies:** Priorizar puntos donde haya aliados.
- **avoid-crossfire:** No atacar puntos donde haya aliados.
- **prioritize-mech:** Priorizar enemigos de tipo `mech`. Si no hay, cualquier otro es válido.
- **avoid-mech:** No atacar enemigos de tipo `mech`.

Consideraciones importantes

- Pueden recibirse **varios protocolos a la vez**.
 - Los protocolos proporcionados **siempre serán compatibles entre sí**.
 - Los puntos situados a **más de 100 metros** deben ignorarse.
 - La solución debe estar diseñada para que **sea fácil añadir nuevos protocolos** en el futuro.
-



Implementación Mobile

La prueba se divide en **dos versiones**.



Versión 1 — MVP (obligatoria)

Objetivo

Implementar la lógica de selección de objetivos y una interfaz mínima para interactuar con ella.

Requisitos funcionales

- La pantalla debe contener:
 1. **Un campo de texto** donde el usuario pueda pegar un JSON de entrada (request).
 2. **Un botón** para procesar la información.
 3. **Una zona de salida** donde se muestre el JSON resultante con el objetivo calculado.
-



Versión 2 — Avanzada (para ir a por nota)

Objetivo

Permitir la creación visual de la petición sin necesidad de escribir JSON manualmente.

Requisitos funcionales

La pantalla debe incluir una sección “**Crear petición**” con:

1. Selección de protocolos

- Lista de **checkboxes** para seleccionar uno o varios protocolos.

2. Gestión de puntos escaneados

- Botón “**Añadir punto**”: Al añadir un punto, se debe mostrar una fila en una **tabla o lista dinámica** con:
 - Campo numérico **X**
Campo numérico **Y**
Selector de tipo de enemigo (**soldier** / **mech**)
 - Campo numérico para el **número de enemigos**
Campo opcional para el **número de aliados**

3. Ejecución

- Botón para calcular el siguiente objetivo usando los datos creados.
 - Visualización del resultado (coordenadas X, Y, Z).
-

Entrega

- Video mostrando el funcionamiento.
- Repositorio público (GitHub, GitLab, etc.)
- README explicando:
 - Decisiones de arquitectura
 - Cómo añadir un nuevo protocolo
 - Cómo ejecutar la app y los tests